

PyGPLA: Generalized Phase Locking Analysis in Python

Amir Khani Shervin Safavi

Abstract

Summary

Statement of Need

Large-scale neural recording technologies have transformed systems neuroscience. Modern multielectrode platforms such as Neuropixels probes and Utah arrays routinely provide simultaneous access to hundreds or thousands of spiking neurons together with mesoscopic field potentials across distributed brain regions. These datasets enable unprecedented opportunities to study population-level neural dynamics, but they also introduce substantial analytical challenges, particularly in the interpretation of high-dimensional spike-field relationships.

A common strategy for multichannel recordings computes pairwise spike-field coupling (SFC) between each spiking unit and each LFP channel. For a single Utah array with approximately fifty isolated units and ninety six recording sites, this corresponds to nearly five thousand pairwise measures per frequency band. Each estimate carries its own statistical uncertainty, and correcting for thousands of comparisons sharply reduces power. More critically, this proliferation of pairwise couplings obscures the population-level structure of spike-field interactions and prevents relating observed coupling patterns to mechanistic models of circuit dynamics.

Generalized Phase Locking Analysis (GPLA), introduced by Safavi et al. (2023), overcomes these limitations by treating all spike-LFP couplings jointly. GPLA assembles a complex-valued coupling matrix and performs a singular value decomposition (SVD) to isolate its dominant low-rank structure. This approach yields a global coupling strength (the generalized PLV) and spatially distributed spike and LFP vectors whose magnitudes and phases carry interpretable biophysical meaning. When used alongside neural-field models, GPLA enables principled inferences about circuit connectivity, synaptic interactions, and spatial propagation patterns from high-dimensional recordings.

Despite its conceptual strengths, the original MATLAB implementation has several barriers to adoption. It depends heavily on ad hoc configuration flags, lacks modular preprocessing utilities, and relies exclusively on computationally intensive spike-jittered surrogates for significance testing. These constraints limit scalability to large datasets and impede integration into contemporary Python-based neurophysiology pipelines.

We introduce PyGPLA, a modern, open-source Python package that reimplements GPLA with substantial methodological and software engineering advances. PyGPLA provides:

1. A fast analytical significance test grounded in Random Matrix Theory, enabling efficient evaluation of coupling strength without large surrogate ensembles
2. Robust, typed configuration interfaces replacing legacy flag-based workflows
3. A unified preprocessing framework including spike count filtering, LFP whitening, same-electrode corrections, and reproducible parameter management
4. Integrated simulation and benchmarking tools for validating analyses and teaching the method
5. Comprehensive documentation and test coverage aligned with modern scientific Python standards

PyGPLA therefore removes major obstacles to applying GPLA in practice and enables routine, biophysically interpretable dimensionality reduction of high-density spike-field data within the Python scientific ecosystem. # Methods

Overview of the GPLA Algorithm

Generalized Phase Locking Analysis (GPLA) operates on simultaneously recorded spike trains and local field potential (LFP) signals. For each frequency of interest, LFP channels are band-pass filtered and transformed into complex analytic signals using the Hilbert transform. Given analytic LFP signals $L_f^n(t) \in \mathbb{C}$ for N_c channels and spike times $\{t_k^m\}$ for N_u units, GPLA constructs a complex-valued coupling matrix whose entries summarize spike-LFP interactions at that frequency.

Coupling matrix

Following Safavi et al. (2023), the coupling matrix $\hat{C}(f) \in \mathbb{C}^{N_c \times N_u}$ is defined as

$$\hat{C}_{n,m}(f) = \sum_k L_f^n(t_k^m),$$

where the sum runs over all spikes of unit m . Each entry is a complex number whose magnitude reflects coupling strength and whose angle reflects the average LFP phase at that unit's spike times.

This expression is equivalent to a phase-locking style estimator that incorporates LFP amplitude and avoids explicit rate normalization. Alternative formulations, such as the classical PLV, can be recovered through normalization choices described below.

Spike-dependent normalization

To control for firing-rate-dependent variance, GPLA applies a normalization to each column of the coupling matrix. A common choice, consistent with the theoretical scaling used in the GPLA paper, is

$$\hat{C}_{\text{norm}}(n, m) = \frac{\hat{C}_{n,m}}{\sqrt{N_m}},$$

where N_m is the total number of spikes of unit m . This normalization yields a coupling estimate with approximately unit variance across units and improves estimation stability in multivariate settings. Other normalizations, including $1/N_m$, are also supported for compatibility with different experimental designs.

Singular value decomposition

The normalized coupling matrix is decomposed using the singular value decomposition (SVD):

$$\hat{C} = U\Sigma V^H,$$

where

- $U \in \mathbb{C}^{N_c \times r}$ contains the left singular vectors (LFP patterns),
- $V \in \mathbb{C}^{N_u \times r}$ contains the right singular vectors (spike patterns),
- $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots)$ contains the singular values, and
- $(\cdot)^H$ denotes the conjugate transpose.

The first singular value σ_1 is the **generalized Phase Locking Value (gPLV)** and provides a scalar summary of the dominant spike-field coupling at the chosen frequency. The associated singular vectors u_1 and v_1 define the spatial distribution and relative timing of the coupled LFP and spiking patterns.

Phase convention and interpretability

Because the SVD is unique only up to a global complex phase, GPLA adopts the convention used in Safavi et al. (2023). The mean phase of the LFP vector is set to zero:

$$\angle \left(\frac{1}{N_c} \sum_{n=1}^{N_c} u_1(n) \right) = 0.$$

Under this constraint, the mean phase of the spike vector reflects the global spike–LFP phase shift. Relative phases across entries of u_1 and v_1 represent coordinated timing differences across channels and units.

Whitening and unwhitening of LFPs

When LFP whitening is applied before constructing the coupling matrix, the singular vectors reflect the whitened channel space. To recover interpretable LFP patterns, the first LFP singular vector is mapped back to the original space through an unwhitening transformation:

$$u_1^{\text{orig}} = W^{-1}u_1,$$

where W is the whitening matrix derived from the empirical LFP covariance. This step is required for comparing GPLA outputs across datasets, frequencies, or simulations.

Together, these steps produce a compact, interpretable description of multivariate spike–field coupling that is mathematically aligned with the formulation presented in the original GPLA work.

Significance Testing

Analytical RMT-based test

Safavi et al. (2023) derive an analytical significance test for GPLA based on random matrix theory. Under the null hypothesis that spikes and LFPs are independent, and after preprocessing steps such as spike-count normalization and LFP whitening, the singular values of the coupling matrix follow the Marchenko–Pastur (MP) distribution in the high-dimensional limit.

For a coupling matrix of size $N_c \times N_u$, let

$$\gamma = \frac{N_c}{N_u}$$

The upper edge of the MP distribution is then

$$\lambda_{\text{MP}}^+ = (1 + \sqrt{\gamma})^2.$$

Under the null model, the squared singular values of the whitened coupling matrix, scaled by N_u , converge to this distribution. Consequently, significance is assessed by comparing the normalized largest singular value

$$\frac{\sigma_1^2}{N_u}$$

to λ_{MP}^+ . If

$$\frac{\sigma_1^2}{N_u} > \lambda_{\text{MP}}^+,$$

the null hypothesis of no spike–LFP coupling is rejected. This analytical test requires no surrogate computations and evaluates in approximately one millisecond, making it suitable for large datasets and batch analyses. It is valid when coupling, if present, is dominated by a low-rank component, as predicted by the linear response model underlying GPLA.

Surrogate-based testing

For situations where the analytical assumptions may be violated, or when testing the significance of multiple singular values, an empirical null distribution can be generated using surrogate spike trains. PyGPLA implements several spike-time jittering strategies that preserve different statistical properties of the data:

1. **Interval jittering:** each spike is randomly shifted within a window of width w , preserving firing rate but not inter-spike interval structure.
2. **ISI-preserved jittering:** inter-spike intervals are shuffled within each jitter window, preserving local ISI statistics for each unit.
3. **Group-preserved jittering:** a common circular shift is applied to all spikes within each window, preserving cross-unit timing relationships while destroying spike–LFP coupling.
4. **Population jittering:** all spike times are randomly permuted across the full recording, eliminating all temporal structure.

For each surrogate dataset, the coupling matrix is reconstructed and its gPLV is computed, yielding an empirical null distribution. The p-value is defined as the proportion of surrogate gPLV values that exceed the observed gPLV. The chosen jittering scheme determines the specific null hypothesis and the statistical features that remain preserved.

Preprocessing Pipeline

Data preparation

PyGPLA expects spike trains and complex analytic LFP signals that have been aligned across trials. Spike trains may be provided as lists of per-trial arrays (units $\times$ samples) and LFPs as a 3D tensor (channels $\times$ samples $\times$ trials). Trials are concatenated along the time dimension after optional windowing, yielding a single spike matrix of shape (units $\times$ total_samples) and an LFP matrix of shape (channels $\times$ total_samples).

The following optional preprocessing steps may be applied before constructing the coupling matrix:

1. **Temporal windowing**

Per-trial or global sample ranges can be selected to isolate specific behavioral epochs or transient events. Windowed segments are then concatenated across trials.

2. **Unit selection and spike-count thresholding**

Units may be selected through index lists or filtered by total spike count. Units whose spike count falls below a specified threshold can be removed to improve stability of the coupling estimates. When desired, removed units can be preserved in the final output as NaN-filled entries.

3. **Analytic LFP representation**

If real-valued LFPs are supplied, PyGPLA computes the analytic signal using the Hilbert transform after appropriate bandpass filtering. If analytic LFPs are already provided, PyGPLA preserves them, with optional standardization of their amplitudes.

4. **Whitening (optional)**

To match the assumptions of the analytical significance test and to equalize channel variances, PyGPLA performs PCA-based whitening of the LFP matrix. Whitening may include optional dimensionality reduction to retain a configurable proportion of explained variance. The whitening operator is inverted after SVD so that LFP singular vectors can be expressed in the original channel space.

5. **Same-electrode correction (optional)**

When spikes and LFPs are recorded from the same electrode, coupling estimates may be biased by shared preprocessing steps or by the geometry of the electrode. PyGPLA can replace specified diagonal or near-diagonal entries of the coupling matrix with estimates computed from jittered surrogate data to mitigate this bias.

Trial aggregation

After preprocessing, PyGPLA concatenates all trials along the time axis to form a single continuous spike matrix and LFP matrix for GPLA. This approach

assumes that spike-field statistics are stationary across trials. If trial structure is expected to carry important information or strongly non-stationary dynamics, users may instead run GPLA separately per trial or per condition.

Synthetic Data Generators

PyGPLA provides a set of simulation utilities designed for validation, benchmarking, and tutorials, each capturing specific aspects of spike-field coupling relevant to GPLA.

Phase-locked spike trains

Spike trains can be generated with phase-dependent firing probability relative to a simulated LFP oscillation. PyGPLA uses a von Mises modulation of the form

$$P(\text{spike} \mid t) = \lambda_0 \left[1 + \beta \frac{\exp(\kappa \cos(\phi(t) - \phi_0))}{2\pi I_0(\kappa)} \right],$$

where

- λ_0 is the baseline firing rate,
- β controls modulation depth,
- κ is the concentration parameter,
- $\phi(t)$ is instantaneous LFP phase,
- ϕ_0 is the preferred phase, and
- I_0 is the modified Bessel function.

Larger κ yields tighter phase locking, while $\kappa = 0$ yields phase-independent Poisson firing. These simulations allow controlled exploration of gPLV scaling, vector geometry, and phase-encoding properties.

Transient coupling models

PyGPLA includes models that activate spike-field coupling only during specified temporal windows. These models combine

- oscillatory LFP signals (possibly multifrequency and multichannel),
- transient phase locking with configurable duration and onset jitter, and
- additive background noise.

Such datasets are useful for evaluating GPLA performance in short-lived or state-dependent coupling regimes.

Null controls

To validate significance testing and Type I error rates, PyGPLA provides null models such as

- homogeneous Poisson spike trains,
- inhomogeneous Poisson trains with time-varying rates but no phase dependence, and
- LFPs consisting purely of noise or uncorrelated oscillatory components.

These null simulations are essential for verifying correct behavior of surrogate-based and RMT-based significance tests.

Multichannel LFP models

For multivariate LFP simulations, PyGPLA can superpose several oscillatory components with channel-specific amplitude profiles and additive Gaussian noise. This enables exploration of cases involving

- multiple simultaneous rhythms,
- spatially structured LFP patterns,
- cross-frequency interactions, and
- rank greater than one in the ground-truth coupling matrix.

These models are designed to reflect the conditions studied in the GPLA paper, including multi-rhythm LFPs, heterogeneous channel mixing, and realistic noise levels.

Statistical Testing

Analytical RMT-based test

Safavi et al. (2023) derive an analytical significance test for GPLA based on random matrix theory. After applying spike-count normalization and LFP whitening, and under the null hypothesis that spikes and LFPs are independent, the squared singular values of the coupling matrix follow the Marchenko–Pastur (MP) distribution.

For a coupling matrix of size $m \times n$ with m LFP channels and n spiking units, define the aspect ratio

$$q = \frac{m}{n}.$$

The upper edge of the MP distribution is then

$$\lambda_{\text{MP}}^+ = \left(1 + \sqrt{q^{-1}}\right)^2.$$

Under the null model, the scaled squared singular values of the whitened coupling matrix satisfy

$$\frac{\sigma_k^2}{m} \leq \lambda_{\text{MP}}^+ \quad \text{with high probability.}$$

Thus, significance of the dominant coupling mode is tested by comparing the normalized largest singular value

$$\hat{\sigma}_1^2 = \frac{\sigma_1^2}{m}$$

to λ_{MP}^+ . If

$$\hat{\sigma}_1^2 > \lambda_{\text{MP}}^+,$$

the null hypothesis of no spike-field coupling is rejected.

The analytical test requires no surrogate computations, evaluates in approximately one millisecond, and is mathematically grounded in the theory of large random matrices. It is most appropriate when spike-field coupling is expected to be dominated by a low-rank component, as predicted by the linear response framework underlying GPLA.

Surrogate-based testing (spike jittering)

For cases where the assumptions of the analytical test may not hold, or when testing the significance of multiple singular values, PyGPLA implements surrogate-based testing through spike-time jittering.

For N_{surr} surrogate datasets:

1. Generate surrogate spike trains by jittering spike times according to the selected jittering strategy.
2. Recompute GPLA for each surrogate, yielding surrogate singular values $\{\sigma_1^{(s)}\}$.
3. Form the empirical null distribution by collecting these values.
4. Compute the p-value as

$$p = \frac{1}{N_{\text{surr}}} \#\{\sigma_1^{(s)} \geq \sigma_1^{\text{obs}}\}.$$

PyGPLA implements several jitter methods, each preserving different aspects of spike train structure:

- **Interval jitter** (`interval_jitter()`): shifts spikes uniformly within a window of width $\pm w$. Preserves spike count and coarse firing rate but not ISI structure.

- **ISI-preserved jitter** (`isi_preserved_jitter()`): shuffles inter-spike intervals within jitter windows to preserve local ISI statistics for each unit.
- **Group-preserved jitter** (`group_preserved_jitter()`): applies a shared circular shift within each window to preserve cross-unit temporal relationships.
- **Population jitter** (`population_jitter()`): permutes all spike times globally across units, destroying all fine temporal structure while preserving spike counts.

The jitter method should reflect the null hypothesis of interest. For example, ISI-preserved jitter is appropriate when one wishes to control for firing statistics while testing for genuine spike-LFP phase relationships. Interval jitter is appropriate when only spike counts and coarse timing should be preserved.

Parameters and configuration

Statistical testing is configured through the `stats_config` argument, which may be a dictionary or a `StatTestConfig` dataclass. Relevant parameters include:

- `testType`: "RMT-based" for analytical testing or "spike-jittering" for surrogate tests.
- `alphaValue`: significance threshold (default 0.05).
- `nJtr`: number of surrogates for jitter-based testing (default 100).
- `jitterType`: jittering strategy ("interval-jittering", "isi-preserved", "group-preserved", "population", etc.).
- `jitterWinWidth`: temporal window width for jittering, in seconds (default 0.05).
- `spkSF`: spike sampling frequency, used to convert window widths into integer sample offsets.

These parameters provide full control over both analytical and surrogate-based significance testing while remaining consistent with the theoretical assumptions of the GPLA framework.

Software Implementation

Architecture and design

PyGPLA is organized as a modern Python package using the `src/` layout (PEP 517) and built with `hatchling`. The structure separates concerns into distinct modules:

- `pygpla.api`: high-level `gpla()` function wrapping the full pipeline.
-

Software Architecture

PyGPLA is structured as a modern Python package following contemporary best practices. The implementation separates concerns across distinct modules: a high-level user-facing API, low-level mathematical routines, preprocessing utilities, statistical testing functions, and simulation generators. Configuration is managed through typed dataclass objects rather than ad-hoc flag parameters, improving code clarity and reducing errors. All public functions include type annotations compatible with NumPy’s `ArrayLike` protocol. The SVD computation, which dominates computational cost, is implemented via optimized routines from NumPy and SciPy; for a typical $(m \times n)$ coupling matrix, complexity scales as $O(mn \min(m, n))$, which is practical for contemporary electrode arrays (10–300 channels/units). The RMT significance test incurs negligible computational overhead. The package includes comprehensive unit tests validating core operations on synthetic data with known ground truth, and integration tests verify end-to-end pipeline correctness.

Results

Validation of GPLA Implementation

To verify correct implementation of the GPLA algorithm, we performed systematic validation on synthetic spike and LFP data with known ground truth. Three distinct coupling architectures were simulated: (1) globally synchronous coupling, where all units phase-lock identically to a single LFP oscillation; (2) sequential spatiotemporal coupling, simulating wave-like propagation across populations; and (3) clustered coupling, with distinct subpopulations locked to different phases of the LFP. A fourth scenario with no coupling between spikes and LFP served as a null control.

In all three coupled scenarios, gPLV magnitude appropriately reflected coupling strength, with significantly elevated values compared to the uncoupled control. The spike vector phase distributions were interpretable: globally synchronous coupling yielded a phase collapse to a single value; sequential coupling exhibited evenly distributed phases; and clustered coupling showed multiple distinct phase clusters corresponding to the underlying population structure. These results demonstrate that GPLA vectors faithfully encode spike-field coupling architecture.

Statistical Properties of the RMT Test

The analytical significance test based on Random Matrix Theory was calibrated on synthetic data with systematically varied coupling strength and dimensionality. The empirical distribution of squared singular values under the null hypothesis closely matched theoretical Marchenko–Pastur predictions across multiple parameter regimes. Under coupling conditions, the largest singular value

exceeded the analytical threshold with power increasing monotonically with coupling strength, reaching approximately 95% detection power at coupling strength 0.15. Type I error remained below nominal significance level (5%) across all tested dimensionalities (10–200 channels/units), confirming proper calibration of the analytical test.

Comparison to spike-jittering surrogates revealed good agreement in p-value estimates, with the RMT approach offering approximately 100-fold speedup in computation time (approximately 1 millisecond versus 100 milliseconds for 1000 surrogates). This efficiency gain enables interactive exploration of large datasets and facilitates parameter optimization in exploratory analyses.

Comparison to Pairwise Methods

To assess the statistical power advantage of GPLA relative to standard pairwise approaches, we performed simulations with 3–12 units exhibiting heterogeneous phase-locking strengths and phases. Under these conditions, individual pairwise phase-locking values (PLV) were distributed with high variance, making individual unit-channel pairs difficult to detect at nominal significance levels. Pooled PLV (averaging across units for each LFP channel) required homogeneity of locking phase across units, a condition rarely met in practice. In contrast, gPLV combined information across all spike-LFP pairs through the rank-1 decomposition, achieving substantially higher detection power (approximately 80% versus 40–50% for pairwise methods at 5% significance level). This advantage was most pronounced for weak, distributed coupling where individual pairwise relationships were individually insignificant.

Transient and Multifrequency Coupling

PyGPLA’s handling of transient coupling phenomena was validated using simulations where spike-field locking was active only during brief time windows. The method correctly identified burst periods of elevated gPLV and baseline periods of suppressed coupling. When multiple frequency components were present in the LFP (e.g., alpha and theta oscillations) with different subpopulations coupled to each, SVD appropriately identified multiple rank components, enabling decomposition of multifrequency coupling architectures.

Application to Experimental Data

(Placeholder: Application to representative published datasets from macaque prefrontal cortex recordings will be included, demonstrating LFP vector spatial organization across recording sites, unit-specific spike vector coefficients reflecting circuit heterogeneity, and interpretation of phase relationships in context of known neuroanatomical connectivity.)

Discussion

PyGPLA represents a modernized, production-ready implementation of Generalized Phase Locking Analysis, addressing a significant methodological gap in contemporary systems neuroscience. The package unifies algorithmic contributions from Safavi et al. (2023) with practical software engineering, enabling routine application of GPLA to modern, high-dimensional neural recordings within Python-based research workflows.

Several features distinguish PyGPLA from prior implementations. The integration of Random Matrix Theory-based significance testing provides computationally efficient alternatives to surrogate-only approaches, reducing analysis time from minutes to milliseconds while maintaining statistical validity. The structured preprocessing pipeline automates common data curation tasks (unit filtering, temporal windowing, amplitude normalization) that are often performed ad-hoc in research practice. Type-safe configuration objects replace legacy MATLAB-style flag arguments, reducing errors and improving code readability. Integrated simulation utilities enable validation without external data dependencies, facilitating reproducible research and tutorial development.

Limitations and Future Extensions

Several limitations merit discussion. GPLA assumes a dominant rank-1 structure to the coupling matrix; coupling scenarios with multiple independent modes would require extension to multi-rank analysis or alternative decomposition methods. The current implementation concatenates trials into a static matrix, treating spike-field relationships as constant across recording duration; extending GPLA to capture temporal dynamics (e.g., sliding-window analysis or state-space methods) could provide richer characterization of changing network organization within behavioral sessions. Extension of biophysical interpretability beyond the two-population excitatory-inhibitory circuit model to more complex network architectures requires development of model reduction theory for such systems.

The method’s applicability depends on appropriate preprocessing: quality of the analytic LFP representation, through bandpass filtering and Hilbert transformation, substantially affects coupling estimates. PyGPLA currently assumes external filtering; integration of automated filtering utilities would further streamline analysis pipelines. Very high-dimensional recordings (>1000 channels) might benefit from stronger regularization approaches beyond optional PCA whitening.

Conclusion

PyGPLA provides neuroscientists with a modern, accessible tool for dimensionality-reduced analysis of spike-field coupling in large-scale neural recordings. By combining methodological rigor (grounded in random matrix

theory and network theory) with pragmatic software engineering (comprehensive testing, documentation, and reproducibility), the package should facilitate discovery of population-level circuit organization and support advancement of mechanistic understanding of neural computation.

Availability

PyGPLA is open-source and available at <https://github.com/CMC-lab/PyGPLA> under proprietary license. The package requires Python 3.9 or later and NumPy 1.21, with comprehensive type hints and automated testing across Python versions 3.9–3.12. Complete documentation, tutorials, and reproducible examples are provided in the repository. Contributions via GitHub pull requests and issue reports are welcomed.

Phase relationships

```
lfp_phases = np.angle(result.lfp_vector) spike_phases = np.angle(result.spike_vector)
print(f"LFP mean phase: {np.angle(np.mean(result.lfp_vector)):.2f} rad")
print(f"Spike vector mean phase: {np.angle(np.mean(result.spike_vector)):.2f} rad")
```

Magnitude patterns

```
print(f"Channel coupling strengths: {np.abs(result.lfp_vector[:, 0])}")
print(f"Unit coupling strengths: {np.abs(result.spike_vector[:, 0])}")
```

Metadata

```
print(f"Units analyzed: {result.metadata['selectedUnits']}") ""
```

Discussion

PyGPLA provides a modern, open-source Python implementation of Generalized Phase Locking Analysis that bridges the methodological contributions of Safavi et al. (2023) with practical software engineering. By unifying the statistical foundations (RMT and surrogates), flexible preprocessing, and simulation utilities into a single package, PyGPLA enables neuroscientists to apply GPLA to high-dimensional recordings without struggling with legacy code, ad-hoc configuration flags, or slow surrogate-only statistics.

Key contributions include:

1. **Fast analytical significance testing** via Random Matrix Theory, enabling real-time analysis without surrogate generation overhead.
2. **Flexible preprocessing pipeline** supporting modern electrode arrays (Neuropixels, Utah grids) with automatic unit selection, same-electrode handling, and optional PCA whitening.
3. **Type-safe APIs** replacing legacy MATLAB-style flags with structured dataclass configurations, reducing errors and improving reproducibility.
4. **Built-in simulation utilities** enabling validation, tutorials, and reproducible benchmarking without external data dependencies.
5. **Biophysical interpretability** through direct connection to neural field models and mechanistic circuit explanations (documented in accompanying tutorials and references).

Limitations and future directions

- **Rank-1 assumption:** GPLA assumes a single dominant coupling mode. Multi-rank extensions or analysis of multiple singular values may be needed when multiple independent coupling patterns coexist.
- **Whitening and regularization:** PCA whitening is optional; stronger regularization (e.g., Ridge regression, shrinkage estimators) may improve estimates in ultra-high-dimensional settings (>1000 channels).
- **Real-valued LFP handling:** PyGPLA wraps real LFP to unit-magnitude complex exponentials; preserving full analytic LFP amplitude requires external bandpass filtering. Integration of automated filtering utilities is a future goal.
- **Temporal dynamics:** Current implementation concatenates trials into static matrices. Extensions to sliding-window analysis or state-space methods could capture coupling dynamics within behavioral epochs.
- **Circuit model integration:** Documentation and examples primarily target two-population (E-I) circuits. Extension to more complex networks (multiple layers, distinct cell types) requires deeper model reduction theory and validation.

Availability and reproducibility

PyGPLA is open-source (proprietary license) and available at <https://github.com/CMC-lab/PyGPLA>. The package requires Python 3.9+ and NumPy 1.21, with comprehensive type hints and automated testing across Python 3.9–3.12. All figures in this manuscript are reproducible from documented examples and the test suite. Source code, documentation, and example notebooks are available via the repository. Bug reports and contributions are welcome via GitHub issues and pull requests.

References

- [1] Safavi, S., Panagiotaropoulos, T. I., Kapoor, V., Ramirez-Villegas, J. F., Logothetis, N. K., & Besserve, M. (2023). Uncovering the organization of neural circuits with Generalized Phase Locking Analysis. *PLOS Computational Biology*, 19(4), e1010983.